5 로봇 제어공학

Robot Control

이 단원을 마치면

짐 무게가 바뀌어도 같은 자리에 서게 하고, 안전하게 힘을 제한하며, 시뮬레이터와 실물의 차이를 보정으로 맞춘다

먼저 배울 것 4 로봇 동역학

15차시 **5-1 피드백의 원리**
필요한 토크를 계산해 그대로 내면 어떨까? 짐을 달아도 원하는 각도에 설까?

16차시 **5-2 PID 제어**
P 제어로는 오차가 조금 남았다. 출렁임 없이 없애려면 무엇을 더해야 할까?

17차시 **5-3 궤적 추종**
목표가 한 점이 아니라 움직이는 궤적이면 어떨까? 관절은 얼마나 늦게 따라갈까?

18차시 **5-4 힘 제한과 안전 여유**
로봇팔이 사람 손에 닿으면 얼마나 세게 누를까?

19차시 **5-5 오차와 캘리브레이션**
같은 명령을 보내도 실제 로봇은 조금 다른 곳에 간다. 왜 시뮬레이터와 다를까?

이 교재 쓰는 법: 한 주제가 한 차시(50분)

도입 학습 목표를 읽고, **생각 열기** 질문에 내 생각을 적어 본 뒤 그림을 봐요.

개념 정의 · 원리 · 자세히를 읽어요. 처음 보는 기호는 위에 작은 이름이 있고, 주제마다 **기호 표**가 있어요. 대학 과정은 수식까지.

탐구 활동 1 → 2 → 3 시뮬레이터 [교육] 탭에서 같은 동작을 한 가지만 바꿔 세 번 해요. 하기 전에 **예상**을 적고, 해 본 뒤 **기록표**를 채우고**왜 그럴까**를 읽어요. `실행` 처럼 네모 친 말은 화면의 버튼 이름이에요.

정리 · 평가 세 결과를 비교해 **기억할 것**과 **현장에서는**을 확인하고, 문항지로 평가해요.

참고 자료 Åström, K. J. & Murray, R. M., Feedback Systems: An Introduction for Scientists and Engineers, 2nd ed., Princeton University Press, 2021 (피드백 원리, PID, 전달함수) · Lynch, K. & Park, F., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017, 11장 (Robot Control) · Siciliano, B., Sciavicco, L., Villani, L. & Oriolo, G., Robotics: Modelling, Planning and Control, Springer, 2009, 8장(Motion Control)·9장(Force Control) · ISO 10218-1, ISO 10218-2 Robotics — Safety requirements for industrial robots · ISO/TS 15066:2016 Robots and robotic devices — Collaborative robots · ISO 9283:1998 Manipulating industrial robots — Performance criteria and related test methods (정확도·반복 정밀도) · Elephant Robotics, myCobot 280 사양 (반복 정밀도 ±0.5 mm, 가반하중 250 g)

15차시

5-1 피드백의 원리

Principle of Feedback (Open Loop and Closed Loop) · 단원 안 1 / 5차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

개루프 제어와 피드백(폐루프) 제어를 비교한다. 차이를 "결과를 재서 보는가" 로 설명할 수 있다.

짐(**외란**)이 생긴 그래프를 읽을 수 있다. 개루프와 **P 제어**의 멈춘 각도를 비교한다. 남은 **오차**도 함께 비교한다.

오차 $e^{\text{오차}}$ = 목표 − 지금 을 계산할 수 있다. **P 제어**에는 오차가 조금 남는다. 그 까닭을 중력 토크와의 균형으로 말한다.

Q

생각 열기

필요한 토크를 계산해 그대로 내면 어떨까? 짐을 달아도 원하는 각도에 설까?

함께 생각해 보기

무거운 가방을 들면 팔이 처지는 것을 우리는 어떻게 알아챌까?

눈을 감고 컵에 물을 따르면 왜 넘치기 쉬울까?

해 볼 것

☐ 눈을 감고 팔을 어깨높이로 든 뒤 눈을 떠 높이를 확인한다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

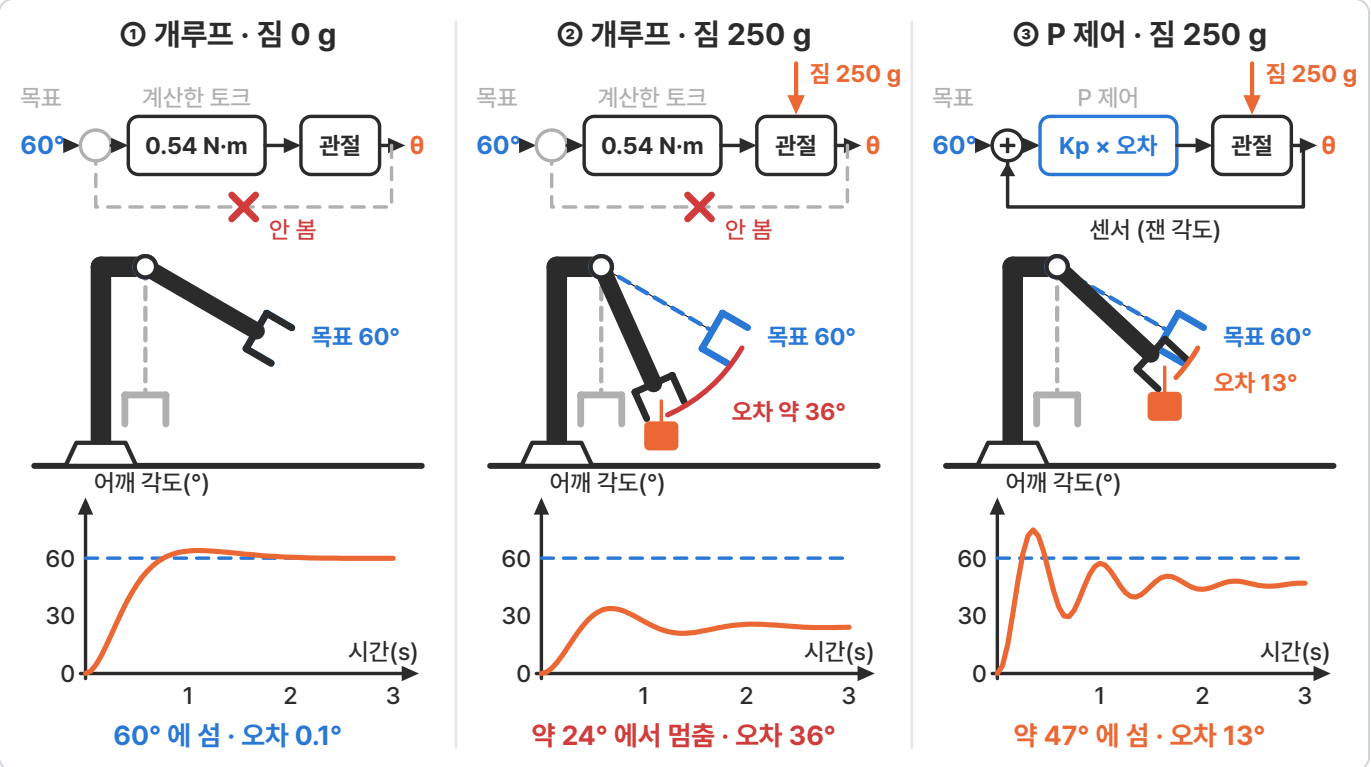


그림 5-1 개루프와 피드백(P 제어)의 어깨 각도: 짐 0 g 개루프 · 짐 250 g 개루프 · 짐 250 g P 제어 (목표 60°) θ 어깨 각도 Kp 비례 게인

정의

개루프 제어는 결과를 보지 않고 정해 둔 명령만 보낸다. 피드백(폐루프) 제어는 센서로 지금 값을 잰다. 목표와의 차이(오차)를 보고 명령을 계속 고친다.

원리

개루프: 짐이 없을 때 필요한 토크를 계산해 그대로 낸다. 모델과 실제가 같으면 맞는다.

짐처럼 모델에 없던 것(**외란**)이 생길 수 있다. 그러면 개루프는 빗나간 줄도 모르고 그대로 멈춘다.

폐루프는 **오차**를 재서 그만큼 힘을 낸다. 오차 e 는 목표 − 지금 이다. 그래서 짐이 생겨도 스스로 고쳐 덜 빗나간다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
θ	세타	**어깨 각도**: 모델의 막대가 아래로 늘어진 자세가 0°, 들어 올릴수록 커진다	°
e	이	**오차**: 목표 각도 − 지금 각도	°
Kp	케이 피	**비례 게인**: 오차에 곱해 토크를 정하는 수. 클수록 세게 민다	N·m/rad (문항에서는 N·m/°)
u	유	**모터 토크**: 제어기가 모터에 내라고 보내는 토크	N·m
m	엠	**질량**: 팔(과 짐)의 질량	kg
g	지	**중력가속도**: 지구가 물체를 끌어당겨 생기는 가속도, 약 9.8	m/s²
l	엘	**무게중심 거리**: 관절 축에서 팔 무게중심까지 거리	m

개념 자세히

4 로봇 동역학에서 팔을 움직이려면 중력·관성·마찰을 이기는 토크가 필요하다는 것을 배웠다. 그러면 필요한 토크를 계산해서 그대로 내면 원하는 자리에 설까? 이 학문은 어깨(J2) 관절 하나만 움직이는 "관절 1개 모델"로 이 질문을 따라간다. 관절 1개 모델은 팔을 축에 매달린 막대 하나로 보고, 관성·마찰·중력 토크와 모터 토크만 남긴 것이다. 어깨 각도 θ(어깨 각도) 는 이 막대가 아래로 늘어진 자세가 0° 이고, 들어 올릴수록 커진다. 화면의 팔은 책상에 닿지 않도록 앞으로 낮게 내민 자세를 0° 로 그리고, 움직임만 보여 준다. 그래서 화면 속 팔 모양과 모델의 막대는 똑같지 않고, 각도와 토크 숫자는 모델로 계산한다. 단순하지만 실제 로봇 관절 제어의 핵심이 다 들어 있다.

결과를 보지 않고 미리 정한 명령만 보내는 것을 **개루프 제어**라고 한다. 짐이 없을 때 60° 에서 버티는 토크(0.54 N·m)를 계산해 두고 그대로 내면, 팔은 조금 넘쳤다가 60° 에 선다. 하지만 250 g 짐을 달면 같은 토크로는 무게를 못 이겨 약 24° 에서 멈춘다. 개루프는 빗나간 것을 모르기 때문에 고칠 수도 없다. 이렇게 짐·마찰 변화처럼 모델에 없던 것이 결과를 흐트러뜨리는 것을 **외란**이라고 한다.

피드백 제어는 센서로 지금 각도를 재서 **오차** e(오차) = 목표 − 지금 을 계산하고, 오차를 줄이는 쪽으로 명령을 고친다. 결과가 다시 명령으로 돌아오는 고리(루프)가 닫혀 있어서 폐루프라고도 부른다. 가장 간단한 **P 제어**는 토크 = Kp(비례 게인) × 오차다. Kp 는 오차 1 rad(약 57°)마다 토크를 몇 N·m 낼지 정하는 수로, 클수록 세게 민다(이 화면은 Kp 4). 짐 250 g 을 단 채로 P 제어를 하면 오차가 클 때 더 세게 밀어 약 47° 까지 올라가고, 남은 오차가 약 36° 에서 13° 로 준다. myCobot 의 **서보 모터** 안에도 각도를 재며 고치는 위치 피드백 제어기가 들어 있다. 실물 myCobot 은 각도 명령만 받으므로, 서보 안의 PID 게인과 토크는 학생이 바꿀 수 없고 이 단원의 토크 · 게인 실험은 시뮬레이터에서 한다.

그래도 13° 가 남는 까닭은, 목표에 가까워질수록 **오차**가 작아져 힘도 약해지다가 중력과 같아지는 곳에서 멈추기 때문이다. 오차가 0 이면 **P 제어**의 힘도 0 이라 무게를 버틸 수 없다. 이 남은 오차를 없애는 방법은 5-2 PID 제어에서 다룬다.

대학 · 수식

관절 1개 모델: $I\,\ddot{\theta} + b\,\dot{\theta} + m\,g\,l\,\sin\theta = u$ (I: 관성, b: 마찰, $m\,g\,l\,\sin\theta$: 중력 토크, u: 모터 토크)

오차 $e(t) = r(t) - y(t)$ (r: 목표, y: 잰 값)

개루프: u = 미리 계산한 값 (짐 없을 때 60° 유지: $u = m\,g\,l\,\sin 60°$)

P 제어: $u = Kp\,e$

P 제어로 멈춘 자리: $Kp\,e = m\,g\,l\,\sin\theta \rightarrow$ 무게가 있으면 $e \neq 0$

탐구 활동 시뮬레이터 [교육] 탭 · 5-1 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 개루프·짐 없음

처음 조건 · 조건: 짐 0 g · 개루프

목표	모델과 실제가 같을 때, 결과를 보지 않고 계산한 토크만으로 목표에 가는지 확인한다.
예상	계산해 둔 토크 0.54 N·m 만 그대로 내면, 팔은 60° 에 설까, 못 미칠까, 넘칠까? ☐ 1. 60° 에 선다 ☐ 2. 못 미친다 ☐ 3. 계속 넘친다
할 일	짐 0 g 을 고르고 개루프 를 누른다. 어깨를 0° → 60° 로 보낸다.
볼 것	계산해 둔 토크 0.54 N·m 만 내는데도 조금 넘쳤다가 60° 에 선다(남은 **오차** 0.1°)
왜	0.54 N·m 는 짐 없는 팔이 60° 에서 받는 중력 토크(m(질량) · g(중력가속도) · l(무게중심 거리) ·sin60°)와 똑같이 계산한 값이다. 올라가는 동안에는 중력 토크가 이보다 작아 남는 토크로 가속하므로 60° 를 조금 지나치지만, 마찰에 속도가 줄며 모터 토크와 중력 토크가 같아지는 60° 로 돌아와 선다(남은 **오차** 0.1°).
핵심	**짐 없음 · 개루프 → 60° 에 선다(오차 0.1°)**

2 활동 2 · 개루프·짐 있음

바꾸는 것 · 짐: 0 g → 250 g

목표	모델에 없던 짐(**외란**)이 생기면 개루프가 어떻게 되는지 알아본다.
예상	같은 토크 0.54 N·m 에 짐 250 g 을 달면, 팔은 몇 도쯤에서 멈출까? 개루프는 빗나간 것을 알아챌까? ☐ 1. 약 60° ☐ 2. 약 47° ☐ 3. 약 24°
할 일	짐 250 g 으로 바꾸고 같은 개루프 를 누른다.
볼 것	같은 토크로는 짐을 못 이겨 약 24° 에서 멈춘다(남은 **오차** 약 36°)
왜	짐을 달면 같은 각도에서도 중력 토크가 훨씬 커져, 0.54 N·m 로는 약 24° 에서 이미 중력 토크와 같아져 더 올라가지 못한다. 개루프는 각도를 재지 않으므로 목표보다 36° 나 모자란 줄 모르고, 명령을 고치지 않은 채 그 자리에 멈춰 있다.
핵심	**짐 250 g 개루프 → 약 24° 에서 멈춘다**

3 활동 3 · 폐루프 P 제어

바꾸는 것 · 제어 방식: 개루프 → P 제어

목표	**오차**를 보고 힘을 고치는 **P 제어**가 짐이 있어도 목표에 얼마나 가까이 가는지 알아본다.
예상	같은 짐 250 g 에서 **P 제어**(Kp(비례 게인) 4)로 바꾸면, 남은 **오차**는 활동 2의 약 36° 보다 줄까? 0 이 될까? ☐ 1. 그대로 약 36° ☐ 2. 줄지만 0 은 아니다 ☐ 3. 0 이 된다
할 일	짐 250 g 그대로 P 제어 를 누른다.
볼 것	**오차**를 보고 힘을 키워 출렁이다가 약 47° 에 선다(남은 오차 13°). 오차가 크게 줄었지만 0 은 아니다
왜	**P 제어**는 토크 u(모터 토크) = Kp × e(오차)라서 **오차**가 클 때 세게 밀어 빨리 올라가고, 넘쳤다가 출렁인다. 목표에 가까워질수록 오차가 작아져 힘도 약해지고, 약 47° 에서 Kp × 13°(약 0.23 rad) ≈ 0.9 N·m 가 짐 단 팔의 중력 토크와 같아져 멈춘다. 오차가 0 이면 P 의 힘도 0 이므로, 무게가 있는 한 오차가 조금 남는다.
핵심	**P 제어 → 약 47° 에 서고 13° 남는다**

✎ 기록표: 화면의 결과를 적어요

활동	짐(g)	멈춘 각도(°)	남은 오차(°)
활동 1 개루프·짐 없음			
활동 2 개루프·짐 있음			
활동 3 페루프 P 제어			

정리: 세 활동 비교하기

활동 1·2·3의 어깨 각도 그래프를 겹쳐 놓는다. 목표 60° 점선도 함께 본다. 짐과 제어 방식에 따라 남은 오차가 어떻게 다른지 본다.

알게 된 것:

정리 질문

짐이 생겨도 덜 빗나가려면 제어기에 무엇이 있어야 할까?

내 답:

기억할 것

개루프는 보지 않고 보낸다. 피드백은 보고 고친다. 그래서 짐이 생겨도 덜 빗나간다.

현장에서는

산업용 로봇과 협동 로봇은 피드백 제어를 한다. 관절마다 엔코더로 각도를 잰다. 서보 드라이브가 1 초에 수천 번 오차를 계산한다. 그 오차로 모터 전류를 고친다. 3D 프린터는 반대로 개루프 장치다. 위치를 재지 않고 스테핑 모터에 정해진 펄스만 보낸다. 그래서 부하가 작고 일정할 때만 쓴다. 걸려서 위치를 놓쳐도(탈조) 스스로 알아채지 못한다.

교사용 답

예상 정답: 활동 1: 1. 60° 에 선다 · 활동 2: 3. 약 24° · 활동 3: 2. 줄지만 0 은 아니다

정리 질문: 지금 각도를 재는 센서와, 목표와의 차이(오차)를 보고 명령을 고치는 피드백이 있어야 한다.

개념 확인 질문: 개루프 제어와 피드백 제어는 무엇이 다를까?

다음 차시 · 16차시 5-2 PID 제어: P 제어로는 오차가 조금 남았다. 출렁임 없이 없애려면 무엇을 더해야 할까?

개루프 제어 open-loop control — 결과를 보지 않고 정해진 명령만 보내는 제어.

피드백 제어 feedback (closed-loop) control — 결과를 재서 목표와의 차이(오차)를 보고 명령을 계속 고치는 제어.

오차 error — 목표와 지금 값의 차이.

외란 Disturbance — 짐 무게·마찰 변화처럼 모델에 없던 것이 결과를 흐트러뜨리는 것

서보 모터 servo motor — 명령한 각도로 스스로 돌아가 멈추는 모터. 안에서 각도를 재며 계속 고친다(피드백).

P 제어 proportional control — 오차에 비례해서 힘을 내는 제어. 오차가 크면 세게, 작으면 약하게.

16차시 5-2 PID 제어

PID Control · 단원 안 2 / 5차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

P · PI · PID 계단 응답 그래프를 읽을 수 있다. **오버슈트** · **정착 시간** · **정상상태 오차**를 찾는다.

P · I · D 가 각각 무엇에 비례해 힘을 내는지 안다. 각 항이 응답을 어떻게 바꾸는지 설명할 수 있다.

게인 튜닝 순서를 말할 수 있다. 게인을 하나씩 더하며 균형을 찾는다. 목표는 "빠르게 · 덜 넘치게 · 오차 없이" 다.

Q **생각 열기**

P 제어로는 오차가 조금 남았다. 출렁임 없이 없애려면 무엇을 더해야 할까?

함께 생각해 보기

샤워기 물 온도를 맞출 때 손잡이를 어떻게 돌릴까?

손잡이를 너무 세게 돌리면 어떤 일이 생길까?

해 볼 것

☐ 샤워기 온도를 맞추는 순서를 세 단계로 적어 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

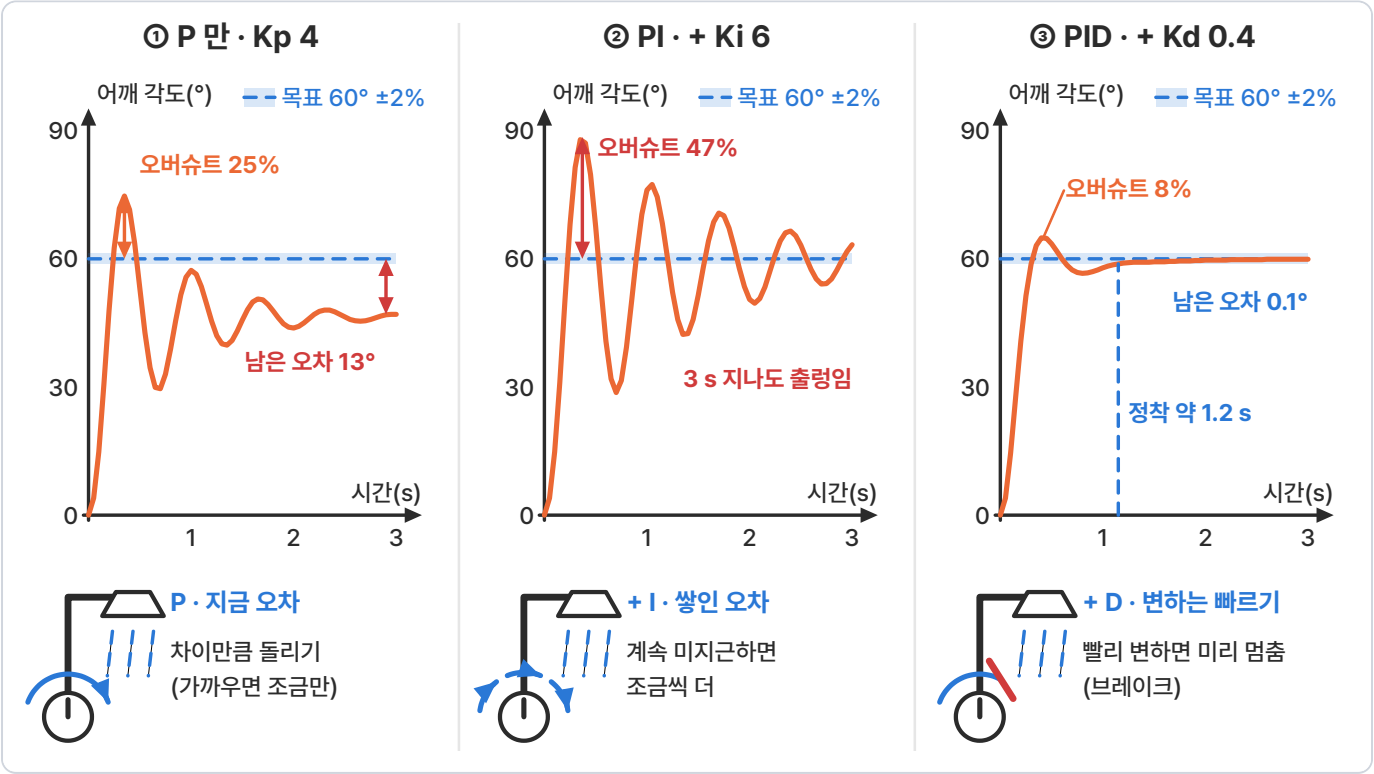


그림 5-2 게인을 하나씩 더할 때의 계단 응답: P 만 · PI · PID (짐 250 g, 목표 60°) Kp 비례 게인 Ki 적분 게인 Kd 미분 게인

정의

PID 제어는 세 가지를 보고 힘을 내는 피드백 제어다. P 는 지금 오차, I 는 쌓인 오차를 본다. D 는 오차가 변하는 빠르기를 본다. 각각에 게인을 곱해 더한 만큼 힘을 낸다.

원리

P 만: Kp (비례 게인)를 키우면 남은 오차가 준다. 하지만 0 이 되지 않고, 넘쳐서 출렁인다(**오버슈트**).

I 를 더하면(PI): 남은 오차가 쌓여 힘을 키운다. 그래서 오차를 없애지만, 더 크게 넘친다.

D 를 더하면(PID): 빨리 다가갈 때 제동을 건다. 그래서 출렁임이 잡히고 빨리 자리 잡는다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
Kp	케이 피	**비례 게인**: 지금 오차에 곱하는 수. 클수록 세게, 빨리 고친다	N·m/rad
Ki	케이 아이	**적분 게인**: 쌓인 오차에 곱하는 수. 남은 오차를 끝까지 없앤다	N·m/(rad·s)
Kd	케이 디	**미분 게인**: 오차가 변하는 빠르기에 곱하는 수. 제동을 걸어 출렁임을 줄인다	N·m·s/rad
e	이	**오차**: 목표 각도 − 지금 각도	°
u	유	**제어 입력**: 제어기가 모터에 내라고 보내는 힘(토크)	N·m
ζ	제타	**감쇠비**: 출렁임이 얼마나 빨리 잦아드는지. 1 이상이면 넘치지 않는다	없음
ω_n	오메가 엔	**고유 진동수**: 응답이 출렁이는 빠르기의 기준	rad/s

개념 자세히

P 제어의 Kp(비례 게인) 처럼 오차에 곱해 힘을 정하는 숫자를 **게인**이라고 한다. 짐 250 g 에서 Kp 를 2 → 4 → 10 으로 키우면 남은 오차는 22° → 13° → 5.5° 로 줄지만 0 이 되지는 않고, 대신 목표를 넘쳐 출렁이는 **오버슈트**가 0% → 25% → 31% 로 커진다. 이 화면은 넘침을 목표 60° 기준으로 잰다. 그래서 Kp 2 는 출렁여도 60° 를 넘지 않아 0% 로 나온다. P 하나만으로는 "빠르고, 넘치지 않고, 오차 없이" 를 모두 얻을 수 없다.

PID 는 세 가지를 더한다. P(비례)는 지금 오차만큼, I(적분)는 지금까지 쌓인 오차만큼, D(미분)는 오차가 변하는 빠르기만큼 힘을 낸다. I 항은 오차가 조금이라도 남아 있으면 계속 쌓여 힘을 키우므로 **정상상태 오차**를 없앤다. 대신 쌓인 힘 때문에 크게 넘쳐서, PI 만 쓰면 **오버슈트**가 약 47% 로 커지고 3 초가 지나도 출렁인다. D 항은 빨리 다가갈 때 미리 제동을 걸어 출렁임을 줄인다. 셋을 다 쓰면(Kp 4, Ki(적분 게인) 6, Kd(미분 게인) 0.4) 오버슈트 약 8%, 약 1.2 초 만에 목표 ±2% 띠 안에 들고 남은 오차는 0.1° 다.

응답이 목표 근처 띠(여기서는 ±2%) 안에 들어가 다시 나오지 않을 때까지 걸린 시간을 **정착 시간**이라고 한다. 좋은 제어는 **오버슈트**·정착 시간·**정상상태 오차**를 함께 작게 만든다. 세 **게인**을 바꿔 가며 이 균형을 찾는 일을 게인 튜닝이라고 한다. 흔히 Kp 로 빠르기를 정하고, Ki 로 남은 오차를 없애고, Kd 로 출렁임을 잡는 순서로 맞춘다. 산업 현장 제어기의 대부분이 PID 다.

대학에서는 제어기를 **전달함수**로 적는다. PID 는 C(s) = Kp + Ki/s + Kd s 다. 관성과 마찰이 있는 관절을 피드백으로 묶으면 근사적으로 2차 시스템처럼 응답하고, 그 모양은 고유 진동수 ω_n 와 감쇠비 ζ 두 숫자로 정해진다. ζ 가 작을수록 **오버슈트**가 크고, $\zeta \geq 1$ 이면 오버슈트가 없다. 이 과정에서는 이름과 간단한 식까지만 다룬다.

대학 · 수식

$u(t) = Kp\, e(t) + Ki \int e\, dt + Kd\, de/dt$

이 화면의 오버슈트(목표 기준) = (최댓값 − 목표) / 목표 × 100%

교과서의 오버슈트(최종값 기준) = (최댓값 − 최종값) / 최종값 × 100% (정상상태 오차가 없으면 위 식과 같다)

PID 전달함수: $C(s) = Kp + Ki/s + Kd\, s$

2차 시스템: $G(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$

교과서 오버슈트(최종값 기준) $Mp = e^{(-\zeta\pi/\sqrt{(1-\zeta^2)})}$ $(0 < \zeta < 1)$, 정착 시간(2%) $\approx 4 / (\zeta\, \omega_n)$

탐구 활동 시뮬레이터 [교육] 탭 · 5-2 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · P 만

처음 조건 · 제어기: P 만 (`Kp` 4)

목표	비례 **게인** 비례 게인 Kp 하나만 바꿀 때 남은 오차와 **오버슈트**가 어떻게 바뀌는지 알아본다.
예상	Kp 를 2 → 4 → 10 으로 키우면 남은 오차와 **오버슈트**는 각각 어떻게 될까? ☐ 1. 둘 다 준다 ☐ 2. 둘 다 커진다 ☐ 3. 오차는 커지고 오버슈트는 준다 ☐ 4. 오차는 줄고 오버슈트는 커진다
할 일	짐 250 g 에서 Kp 4, Ki·Kd 0 으로 실행 한다. Kp 를 2·10 으로도 바꿔 본다.
볼 것	Kp 4 에서 출렁이다가 약 47° 에 서서 13° 가 남는다. Kp 를 키우면 남은 오차는 줄지만 **오버슈트**가 커진다
왜	P 는 오차에 비례해 힘을 내므로, 팔이 서는 곳은 Kp × 오차가 짐 단 팔의 중력 토크와 같아지는 자리다. Kp 가 클수록 같은 힘을 더 작은 오차로 낼 수 있어 남은 오차가 22° → 13° → 5.5° 로 준다. 대신 처음에 더 세게 밀어 빨리 달려드니 목표를 지나쳐, **오버슈트**는 0% → 25% → 31% 로 커진다. Kp 2 도 그래프는 출렁이지만, 60° 를 넘지 않아 목표 기준 오버슈트는 0% 다.
핵심	**Kp 를 키움 → 오차는 줄고 더 넘친다**

2 활동 2 · PI

바꾸는 것 · 제어기: P → PI (`Ki` 6 더함)

목표	I 항이 남은 오차를 어떻게 없애는지, 그 대가로 무엇이 생기는지 알아본다.
예상	Kp 4 에 적분 게인 Ki 6 을 더하면, 13° 남던 오차는 어떻게 될까? 출렁임은 줄까, 커질까? ☐ 1. 오차가 줄지만 더 크게 출렁인다 ☐ 2. 오차도 출렁임도 준다 ☐ 3. 오차가 그대로 13° 다
할 일	Kp 4 에 Ki 를 6 으로 올려 실행 한다.
볼 것	쌓인 오차가 밀어 올려 남은 오차가 줄지만, 크게 넘치고(**오버슈트** 약 47%) 3 초가 지나도 출렁인다
왜	I 항은 오차를 시간에 따라 쌓아(적분) 힘을 키우므로, 오차가 조금이라도 남아 있으면 점점 더 세게 민다. 그래서 47° 에서 멈추지 않고 목표까지 밀어 올린다. 하지만 올라가는 동안 쌓인 힘은 목표를 지나도 바로 사라지지 않아 크게 넘치고(약 47%), 반대쪽 오차가 다시 쌓여 힘을 줄일 때까지 3 초가 지나도 출렁인다.
핵심	**I 더함 → 오차는 줄지만 47% 넘쳐 출렁인다**

3 활동 3 · PID

바꾸는 것 · 제어기: PI → PID (`Kd` 0.4 더함)

목표	D 항이 출렁임을 어떻게 잡는지 확인하고, 세 **게인**을 함께 쓴 응답의 세 값을 읽는다.
예상	미분 게인 Kd 0.4 를 더하면 **오버슈트**와 **정착 시간**은 활동 2(PI)보다 어떻게 바뀔까? ☐ 1. 둘 다 커진다 ☐ 2. 오버슈트는 줄고 정착 시간은 길어진다 ☐ 3. 둘 다 줄어든다
할 일	Kp 4, Ki 6 에 Kd 를 0.4 로 올려 실행 한다.
볼 것	출렁임이 잡혀 **오버슈트** 약 8%, 약 1.2 초 만에 ±2% 띠 안에 들고 남은 오차 0.1°
왜	D 항은 오차가 변하는 빠르기에 Kd 를 곱해, 목표로 빨리 다가갈수록 반대 방향 힘(제동)을 낸다. 그래서 넘치기 전에 속도가 줄어 **오버슈트**가 47% → 8% 로 줄고, 출렁임이 빨리 잦아들어 약 1.2 초에 ±2% 띠(60° ± 1.2°) 안에 든다. I 항은 그대로 있으므로 남은 오차도 0.1° 로 거의 없다.
핵심	**D 더함 → 8% 만 넘치고 약 1.2 초에 선다**

✎ 기록표: 화면의 결과를 적어요

활동	오버슈트(%)	정착 시간(s, ±2% 띠)	남은 오차(°)
활동 1 P 만			
활동 2 PI			
활동 3 PID			

정리: 세 활동 비교하기

P·PI·PID 응답을 한 그래프에 겹쳐 놓는다. 게인을 하나씩 더하며 세 값을 표로 본다. 세 값이 어떻게 바뀌는지 비교한다.

알게 된 것:

정리 질문

빠르고, 덜 넘치고, 오차 없이 서려면 세 게인을 어떻게 맞출까?

내 답:

기억할 것

P 는 빠르게 고친다. I 는 끝까지 고친다. D 는 출렁임 없이 고친다. 그래서 세 게인을 함께 맞춘다.

현장에서는

현장 제어기의 대부분이 PID 다. 산업용 로봇 관절의 서보 드라이브도 그렇다. 공장의 온도 · 압력 · 유량 조절기도 그렇다. 로봇을 설치하거나 공구를 바꾸면 게인을 다시 맞춘다. 엔지니어가 짐과 속도에 맞춰 맞춘다. 많은 서보 드라이브에는 자동 튜닝 기능이 있다. 응답을 재서 게인을 스스로 찾는 기능이다.

교사용 답

예상 정답: 활동 1: 4. 오차는 줄고 오버슈트는 커진다 · 활동 2: 1. 오차가 줄지만 더 크게 출렁인다 · 활동 3: 3. 둘 다 줄어든다

정리 질문: *Kp* 로 빠르기를 정하고, *Ki* 로 남은 오차를 없애고, *Kd* 로 출렁임을 잡는다.

개념 확인 질문: P · I · D 는 각각 무엇을 보고 힘을 낼까?

다음 차시 · 17차시 5-3 궤적 추종: 목표가 한 점이 아니라 움직이는 궤적이면 어떨까? 관절은 얼마나 늦게 따라갈까?

P 제어 proportional control — 오차에 비례해서 힘을 내는 제어. 오차가 크면 세게, 작으면 약하게.

게인 Gain — 오차(또는 쌓인 오차·변화율)에 곱해 힘을 정하는 숫자. Kp·Ki·Kd

정상상태 오차 steady-state error — 충분히 기다려도 남아 있는 오차.

전달함수 Transfer function — 입력과 출력의 관계를 s 의 식으로 적은 것. 대학 과정에서 제어기·시스템을 나타낼 때 쓴다

PID 제어 PID control — 오차(P) + 쌓인 오차(I) + 오차가 바뀌는 빠르기(D)를 함께 보는 제어. 가장 널리 쓰인다.

오버슈트 overshoot — 목표를 지나쳤다가 되돌아오는 것.

정착 시간 Settling time — 응답이 목표 근처 띠(예: ±2%) 안에 들어가 다시 나오지 않을 때까지 걸린 시간

17차시 5-3 궤적 추종

Trajectory Tracking · 단원 안 3 / 5차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

명령 **궤적**과 실제 궤적 그래프에서 **추종 오차**를 읽는다. 피드백만 쓰면 실제가 뒤처진다(**서보 지연**). 그 까닭을 설명할 수 있다.

같은 **궤적**을 빨리 갈수록 **추종 오차**가 커진다. 그 까닭을 필요한 토크로 설명할 수 있다. 빨리 갈 때 더 드는 토크는 관성 × 각가속도다.

피드포워드가 미리 계산해 더하는 것을 말할 수 있다. 피드백과 함께 쓰는 까닭을 설명할 수 있다.

Q **생각 열기**

목표가 한 점이 아니라 움직이는 궤적이면 어떨까? 관절은 얼마나 늦게 따라갈까?

함께 생각해 보기

앞사람을 따라 걸을 때 왜 늘 한 걸음 늦을까?

앞사람이 갑자기 뛰면 차이는 어떻게 될까?

해 볼 것

☐ 짝이 손을 천천히, 또 빠르게 움직이면 그대로 따라 해 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

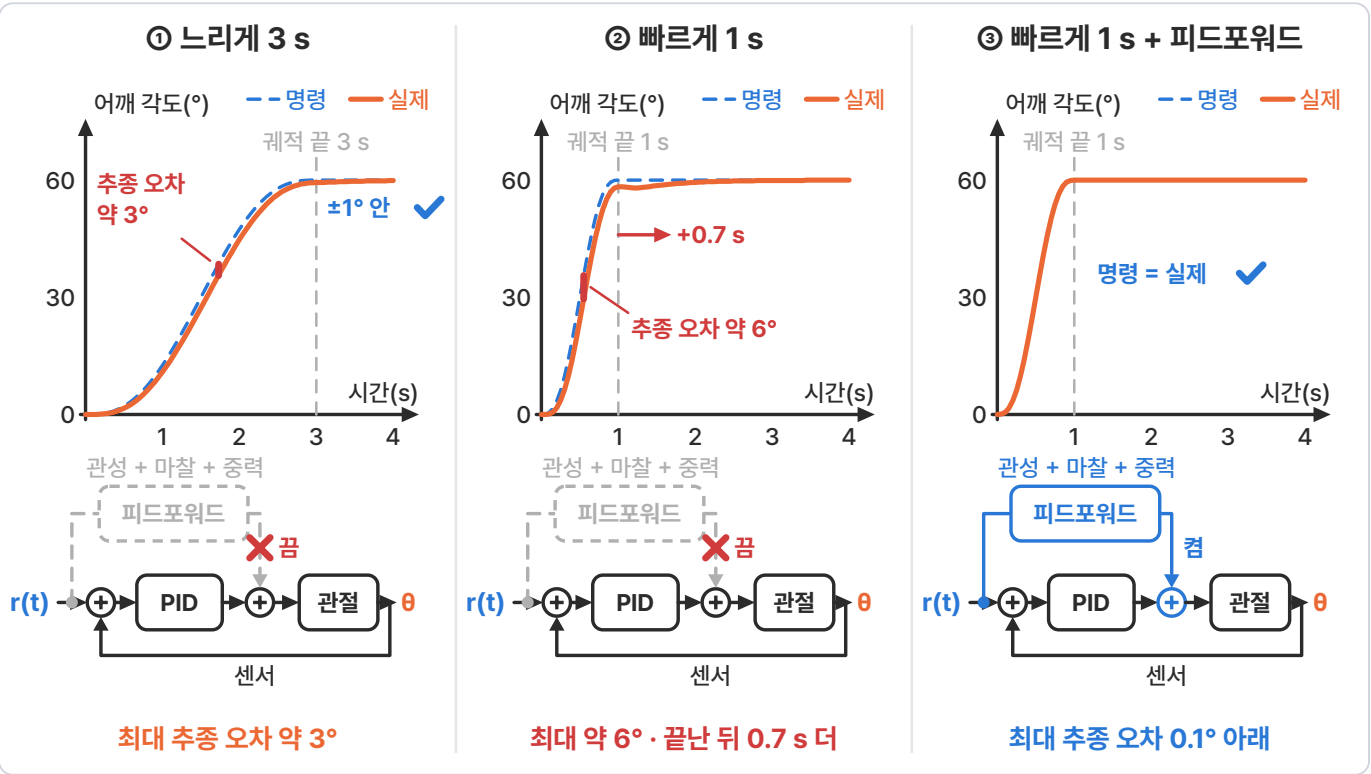


그림 5-3 궤적을 따라가는 어깨 각도와 추종 오차: 3 s · 1 s · 1 s + 피드포워드 (0° → 60°) r 명령 각도 θ 실제 각도

정의

명령 궤적은 시간마다 정해져 있다. 궤적 추종은 관절이 이 궤적을 따라가게 하는 것이다. 이때 명령과 실제의 차이를 추종 오차라고 한다.

원리

피드백은 오차가 생긴 뒤에야 힘을 낸다. 그래서 실제가 명령보다 늘 조금 뒤처진다(**서보 지연**).

같은 **궤적**을 빠르게 가면 관성을 이길 힘이 더 든다. 그래서 **추종 오차**가 커진다.

필요한 토크를 동역학으로 미리 계산해 더한다. 이것이 **피드포워드**다. 그러면 명령과 실제가 거의 겹친다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
r	알	**명령 각도**: 그 시각에 팔이 있어야 할 각도. 시간에 따라 바뀐다	°
θ	세타	**실제 각도**: 센서로 잰 지금 어깨 각도	°
e	이	**추종 오차**: 명령 각도 − 실제 각도	°
$\dot{e}$	이 닷	**오차 속도**: 추종 오차가 변하는 빠르기	°/s
$\ddot{e}$	이 투닷	**오차 가속도**: 오차 속도가 변하는 빠르기	°/s²
Kp	케이 피	**비례 게인**: 지금 오차에 곱하는 수(5-2)	N·m/rad
Ki	케이 아이	**적분 게인**: 쌓인 오차에 곱하는 수(5-2)	N·m/(rad·s)
Kd	케이 디	**미분 게인**: 오차가 변하는 빠르기에 곱하는 수(5-2)	N·m·s/rad
Δ	델타	**이동 각**: 궤적이 처음부터 끝까지 움직이는 각도	°
T	티	**궤적 시간**: 궤적을 처음부터 끝까지 가는 데 걸리는 시간	s

개념 자세히

지금까지는 목표가 한 점(60°)이었다. 실제 로봇은 3 미분기구학 · **궤적** 생성에서 만든 궤적, 곧 "몇 초에 몇 도에 있어야 하는지" 를 정한 명령을 따라가야 한다. 시간에 따라 바뀌는 명령을 따라가게 하는 것을 **궤적 추종**이라고 한다. 그 시각의 명령 각도를 r(명령 각도), 실제 어깨 각도를 θ(실제 각도)라 하면, 둘의 차이 e(추종 오차) = $r - \theta$ 가 **추종 오차**다. 이 주제는 5-2 의 PID(Kp(비례 게인) 4, Ki(적분 게인) 6, Kd(미분 게인) 0.4)를 그대로 두고, 짐 없이 어깨를 0° → 60° 부드러운(5차) 궤적으로 움직인다.

피드백 제어는 오차가 생긴 뒤에야 힘을 낸다. 그래서 움직이는 명령을 따라갈 때는 실제가 늘 조금 뒤처진다. 이것을 **서보 지연**이라고 부른다. 같은 **궤적**을 3 초에 가면 최대 **추종 오차**가 약 3° 이고 궤적이 끝날 때 이미 목표 ±1° 안에 있지만, 1 초에 가면 최대 추종 오차가 약 6° 로 커지고 궤적이 끝난 뒤에도 0.7 초쯤 더 걸려서야 ±1° 안에 든다. 빠를수록 관성을 이기는 토크가 많이 필요한데, 피드백은 그 토크를 오차가 커진 뒤에야 내기 때문이다.

피드포워드는 오차를 기다리지 않고 필요한 토크를 미리 계산해 더하는 방법이다. 4 로봇 동역학에서 배운 관성 토크(관성 × 각가속도), 마찰, 중력 토크를 명령 **궤적**으로 계산해 넣으면, 피드백은 남은 작은 차이만 고치면 된다. 시뮬레이터에서는 모델이 실제와 똑같기 때문에 1 초 궤적도 오차가 0.1° 아래로 따라간다. 실제로는 모델이 조금씩 틀리므로 피드포워드로 대부분을 내고, 남은 차이는 피드백이 고치게 함께 쓴다.

여러 관절에 대해 4 로봇 동역학의 운동 방정식 전체(관성 행렬·원심력과 코리올리 항·중력 토크)로 이 계산을 하는 것을 **계산 토크 제어**라고 한다. 이름만 알아 둔다. 산업용 로봇이 빠르면서도 정확하게 **궤적**을 따라가는 것은 이런 모델 기반 계산 덕분이다.

대학 · 수식

추종 오차 $e(t) = r(t) - \theta(t)$ ($r(t)$: 시간에 따라 바뀌는 명령)

피드백만: $u = Kp\, e + Ki \int e\, dt + Kd\, \dot{e}$

피드포워드 + 피드백: $u = [I\, \ddot{r} + b\, \dot{r} + m\, g\, l \sin r] + Kp\, e + Ki \int e\, dt + Kd\, \dot{e}$

5차 궤적: $r(t) = r_0 + \Delta\,(10s^3 - 15s^4 + 6s^5)$, $s = t/T$ → 최대 속도 $1.875\,\Delta/T$, 최대 가속도 $\approx 5.77\,\Delta/T^2$

계산 토크 제어(이름만): $u = M(\theta)(\ddot{r} + Kd\,\dot{e} + Kp\, e) + C(\theta, \dot{\theta})\dot{\theta} + \tau_g(\theta)$ (여기의 $Kp \cdot Kd$ 는 단위가 $1/s^2 \cdot 1/s$ 인 게인) → 오차가 $\ddot{e} + Kd\,\dot{e} + Kp\, e = 0$ 을 따른다

탐구 활동 시뮬레이터 [교육] 탭 · 5-3 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 느리게 — 처음 조건 · 궤적 시간: 3 초

목표	피드백(PID)만으로 움직이는 명령을 따라갈 때 얼마나 뒤처지는지 잰다.
예상	명령이 3 초 동안 0° → 60° 로 부드럽게 바뀌면, 실제 팔은 명령보다 앞설까, 뒤처질까? 차이는 몇 도쯤일까? ☐ 1. 약 3° 뒤처진다 ☐ 2. 앞선다 ☐ 3. 딱 겹친다
할 일	느리게 3 초 를 누른다. 어깨가 0° → 60° 명령 **궤적**을 따라간다.
볼 것	실제(실선)가 명령(점선)을 조금 늦게 따라가고, 최대 **추종 오차**는 약 3° 이며 **궤적**이 끝날 때 이미 목표 ±1° 안이다
왜	피드백은 오차가 생겨야 힘을 낸다. 팔을 들어 올리고 가속하는 데 드는 토크를 내려면 먼저 명령보다 조금 뒤처져 오차가 생겨야 하므로, 실제는 늘 명령 뒤를 따라간다. 3 초 **궤적**은 느려서 필요한 토크가 작으니 오차도 작고(최대 약 3°), 궤적이 끝날 때 이미 목표 ±1° 안에 있다.
핵심	**3 초 궤적 → 최대 약 3° 뒤처진다**

2 활동 2 · 빠르게 — 바꾸는 것 · 궤적 시간: 3 초 → 1 초

목표	같은 **궤적**을 빨리 갈 때 **추종 오차**가 어떻게 바뀌는지 알아본다.
예상	**궤적** 시간을 3 초에서 1 초로 줄이면 최대 **추종 오차**는 커질까, 작아질까? 궤적이 끝난 뒤에도 따라잡는 데 시간이 걸릴까? ☐ 1. 작아진다 ☐ 2. 그대로다 ☐ 3. 커지고 끝난 뒤에도 더 걸린다
할 일	같은 **궤적**을 빠르게 1 초 로 따라가게 한다.
볼 것	실제가 더 많이 뒤처져 최대 **추종 오차**가 약 6° 로 커지고, **궤적**이 끝난 뒤 0.7 초쯤 더 걸려서야 ±1° 안에 든다
왜	시간이 1/3 이 되면 필요한 각가속도는 9 배(5차 **궤적**의 최대 가속도 ≈ 5.77 Δ(이동 각궤적 시간) / T^2)가 되어, 관성을 이기는 토크가 훨씬 커진다. 피드백은 그 큰 토크를 오차가 커진 뒤에야 내므로 최대 **추종 오차**가 약 6° 로 커지고, 명령이 멈춘 뒤에도 뒤처진 만큼 따라잡느라 0.7 초쯤 더 걸린다.
핵심	**1 초 궤적 → 약 6° 뒤처지고 0.7 초 더 걸린다**

3 활동 3 · 빠르게 + 피드포워드 — 바꾸는 것 · 피드포워드: 끔 → 켬

목표	필요한 토크를 미리 더하는 **피드포워드**가 **추종 오차**를 얼마나 줄이는지 확인한다.
예상	같은 1 초 **궤적**에서 관성 + 마찰 + 중력 토크를 미리 계산해 더하면, 실제 팔은 명령과 얼마나 가까워질까? ☐ 1. 그대로 약 6° 뒤처진다 ☐ 2. 거의 겹친다 ☐ 3. 더 벌어진다
할 일	빠르게 1 초 에서 피드포워드 를 켠다. 그리고 다시 누른다.
볼 것	미리 계산한 토크(관성 + 마찰 + 중력)를 더하니 실제가 명령과 거의 겹친다(최대 **추종 오차** 0.1° 아래)
왜	**피드포워드**는 명령 **궤적** r(명령 각도)로 그 순간 필요한 토크(관성 × 각가속도 + 마찰 + 중력)를 계산해, 오차가 생기기 전에 미리 낸다. 시뮬레이터에서는 이 모델이 실제와 똑같아 필요한 토크가 거의 다 미리 들어가므로, 피드백이 고칠 것이 없어 최대 **추종 오차**가 0.1° 아래로 준다. 실물에서는 모델이 조금씩 틀리므로 남은 차이를 피드백이 고친다.
핵심	**피드포워드 → 명령과 거의 겹친다(0.1° 아래)**

✎ 기록표: 화면의 결과를 적어요

활동	궤적 시간(s)	최대 추종 오차(°)	궤적이 끝난 뒤 ±1° 안에 들기까지 걸린 시간(s)
활동 1 느리게			
활동 2 빠르게			
활동 3 빠르게 + 피드포워드			

정리: 세 활동 비교하기

활동 1·2·3의 추종 오차 그래프를 겹쳐 놓는다. 빠르기와 피드포워드에 따라 오차를 비교한다.

알게 된 것:

정리 질문

빠른 궤적을 정확히 따라가려면 피드백에 무엇을 더해야 할까?

내 답:

기억할 것

피드백만으로는 늘 조금 늦다. 빠른 궤적은 필요한 토크를 미리 계산해 더한다. 이것이 피드포워드다. 남은 차이는 피드백이 고친다.

현장에서는

용접 · 도장 · 실링 로봇은 정해진 경로를 따라간다. 속도도 일정해야 한다. 이런 작업에서는 추종 오차가 곧 불량이 된다. 용접선이 비뚤어지고 도막이 고르지 않게 된다. 산업용 로봇 제어기는 피드백에 피드포워드를 더한다. 동역학 모델로 필요한 토크를 미리 계산해 넣는다. 무거운 공구를 달면 모델도 맞춰야 한다. 공구의 질량 · 무게중심 값을 제어기에 입력한다.

교사용 답

예상 정답: 활동 1: 1. 약 3° 뒤처진다 · 활동 2: 3. 커지고 끝난 뒤에도 더 걸린다 · 활동 3: 2. 거의 겹친다

정리 질문: 필요한 토크를 미리 계산해 더하는 피드포워드를 쓰고, 남은 차이는 피드백이 고친다.

개념 확인 질문: 추종 오차는 무엇과 무엇의 차이일까?

다음 차시 · 18차시 5-4 힘 제한과 안전 여유: 로봇팔이 사람 손에 닿으면 얼마나 세게 누를까?

궤적 trajectory — 경로 + 시간. 언제 어디에 있는지까지 정한 움직임.

추종 오차 Tracking error — 같은 시각의 명령 궤적과 실제 궤적의 차이

피드포워드 Feedforward — 오차를 기다리지 않고 모델로 필요한 토크를 미리 계산해 명령에 더하는 것

궤적 추종 Trajectory tracking — 시간에 따라 바뀌는 명령 궤적을 실제 관절이 따라가게 하는 것

서보 지연 Servo lag — 피드백이 오차가 생긴 뒤에야 힘을 내서 실제가 명령보다 뒤처지는 것

계산 토크 제어 Computed torque control — 동역학 모델 전체로 필요한 토크를 계산해 넣고 남은 오차를 피드백으로 고치는 제어. 이름만

18차시 5-4 힘 제한과 안전 여유

Force Limiting and Safety Margin · 단원 안 4 / 5차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

팔이 막혔을 때 계속 누르는 토크를 구할 수 있다. "**토크 제한** − 중력 토크" 로 구한다. $F = \tau \div r$ (누르는 힘 = 누르는 토크 ÷ 팔 길이) 을 쓸 수 있다.

토크 제한을 낮추면 안전과 성능이 맞바뀐다. 그 관계를 설명할 수 있다. 안전은 덜 누르는 것이다. 성능은 도착 시간 · 버틸 수 있는 무게다.

협동 로봇의 힘 제한 · **충돌 감지**를 말할 수 있다. 관련 표준은 ISO 10218 · ISO/TS 15066 이다. 두 표준이 하는 일도 말할 수 있다.

Q 생각 열기

로봇팔이 사람 손에 닿으면 얼마나 세게 누를까?

함께 생각해 보기

엘리베이터 문이 닫히다가 손에 닿으면 어떻게 될까?

로봇팔이 사람 손을 누른다면 얼마나 세게 눌러야 위험할까?

해 볼 것

- ☐ 손바닥으로 책상을 약하게, 또 세게 눌러 힘의 차이를 느껴 본다
- ☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

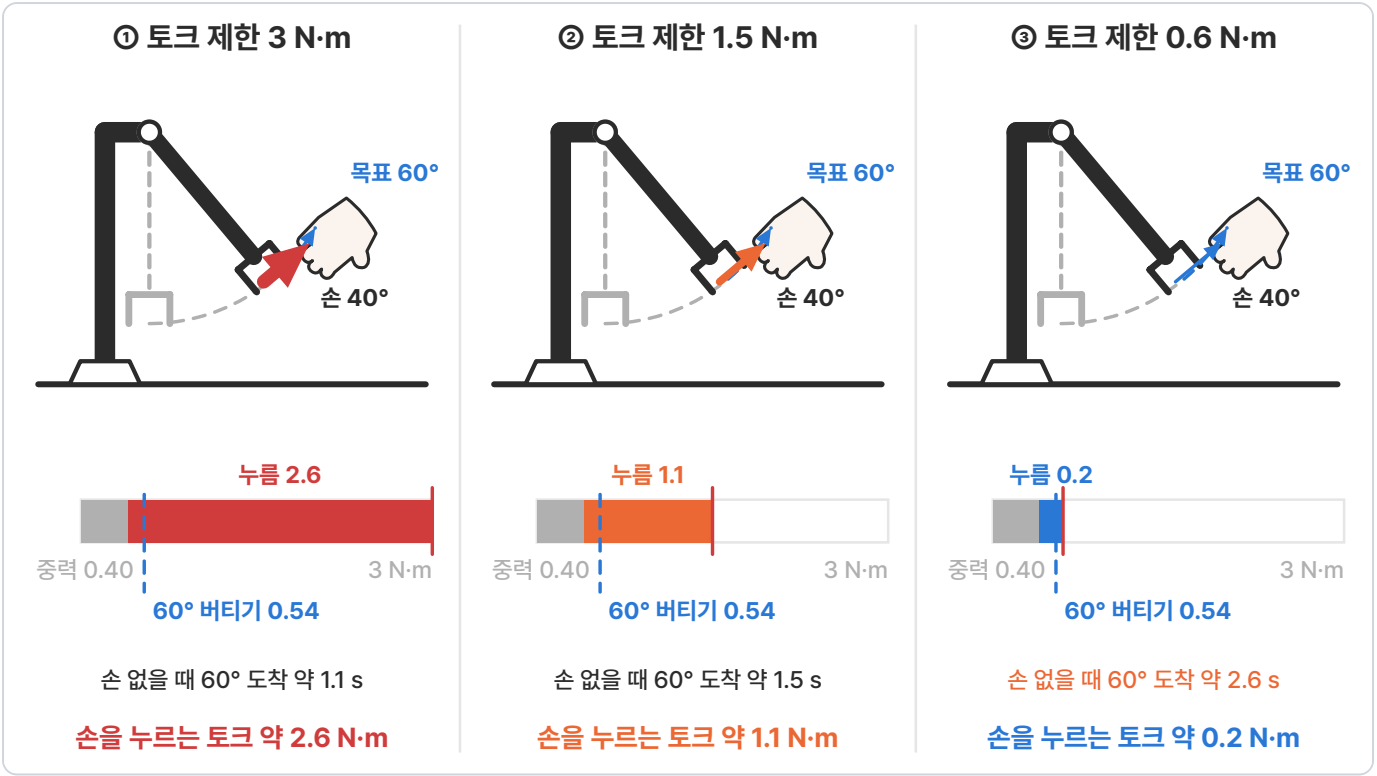


그림 5-4 토크 제한에 따라 손을 누르는 토크: 3 · 1.5 · 0.6 N·m (손 40° 자리)

정의

힘 제한은 모터 토크에 상한을 두는 것이다. 그러면 무언가에 닿았을 때 누르는 힘이 준다. 상한은 작업에 필요한 토크보다 조금만 크게 둔다. 이 차이를 작업 여유라고 한다. 누르는 힘은 사람 몸의 한계보다 넉넉히 작아야 한다. 이 차이가 안전 여유다.

원리

팔이 손에 막히면 오차가 남는다. 제어기는 힘을 계속 키운다. 모터는 **토크 제한**값까지 민다.

누르는 토크는 **토크 제한**에서 중력 토크를 뺀 값이다. 중력 토크는 닿은 자리의 값이다. 그래서 제한을 낮추면 누르는 힘이 준다.

너무 낮추면 무게를 버티지 못하고 느려진다. 그래서 제한은 "필요한 토크 + **작업 여유**" 로 정한다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
τ	타우	**누르는 토크**: 팔이 막힌 뒤 손을 계속 누르는 토크	N·m
F	에프	**누르는 힘**: 닿은 점에서 손이 받는 힘	N
r	알	**팔 길이**: 관절 축에서 닿은 점까지 거리	m
K	케이	**강성**: 관절을 스프링처럼 무르게 할 때의 단단함. 작을수록 무르다	N·m/rad

개념 자세히

사람과 같은 공간에서 일하는 로봇을 **협동 로봇**이라고 한다. 협동 로봇은 울타리 없이 사람 곁에서 움직이므로, 부딪혀도 다치지 않을 만큼 힘과 속도를 제한해야 한다. 국제 표준 ISO 10218 은 산업용 로봇의 안전 요구 사항을 정하고, 기술 규격 ISO/TS 15066 은 협동 로봇이 사람과 닿을 때 몸 부위마다 견딜 수 있는 힘·압력의 한계를 정리해 두었다. 이 과정에서는 표준의 이름과 하는 일까지만 안다. myCobot 280 은 ISO 10218 · ISO/TS 15066 에 맞춘 안전 인증을 받은 협동 로봇이 아니다. 그래서 실습의 **안전 여유**는 사람이 정한 규칙(움직이는 팔에 손 넣지 않기)과 소프트웨어의 속도 · 범위 제한으로 지킨다.

관절 1개 모델은 원리를 보기 위한 단순한 팔 하나다. **토크 제한** 3 · 1.5 · 0.6 N·m 는 이 모델의 값이다. myCobot 280 어깨(J2)의 한계는 4-1 에서 본 약 1.57 N·m 인데, 서보의 정격 토크가 공개되지 않아 시뮬레이터가 추정한 값이다. 이 모델에서 팔이 0° → 60° 로 올라가는 길(40° 자리)에 손 모형을 둔다. 팔이 손에 닿으면 목표까지 오차가 남아 있으므로 PID 는 계속 힘을 키우고, 결국 모터는 토크 제한값까지 민다. 이때 손을 계속 누르는 토크는 토크 제한에서 그 자리의 중력 토크(약 0.40 N·m)를 뺀 만큼이다. 제한을 3 → 1.5 → 0.6 N·m 로 낮추면 누르는 토크는 약 2.6 → 1.1 → 0.2 N·m 로 준다. 닿은 점에서 손이 받는 힘 F(누르는 힘) 는 이 누르는 토크 τ(누르는 토크) 를 관절에서 닿은 점까지의 거리 r(팔 길이) 로 나눈 값이다($F = \tau \div r$). 이 실습에서 r 은 약 0.31 m 라, 누르는 힘은 약 8.3 → 3.5 → 0.6 N 이 된다.

그렇다고 제한을 마냥 낮출 수는 없다. 4-1 에서 본 것처럼 팔은 버티는 데만도 토크가 필요하다. 이 모델에서 60° 에서 버티려면 0.54 N·m 가 필요해, 제한이 0.6 N·m 이면 남는 힘이 거의 없다. 그래서 손이 없을 때 60° 에 자리 잡는 시간(±2% 안에 들기까지)도 약 1.1 초에서 약 2.6 초로 늘고, 짐을 조금만 달아도 버티지 못한다. **토크 제한**은 작업에 필요한 토크보다 조금 큰 값, 곧 "필요한 토크 + **작업 여유**" 로 정한다. 작업 여유를 크게 잡으면 일은 잘하지만 닿았을 때 더 세게 누르므로, **안전 여유**는 그만큼 준다. 실제 **협동 로봇**은 여기에 더해, 토크나 오차가 갑자기 커지면 무언가에 부딪힌 것으로 보고 멈추거나 물러나는 **충돌 감지**를 함께 쓴다.

안전 여유는 힘에만 있는 것이 아니다. 로봇과 장애물 사이 거리에도 여유를 두어 닿기 전에 미리 느려지거나 멈추게 하는데, 이 거리 여유와 충돌 검사는 6 동작 계획에서 다시 만난다. 또 손끝을 스프링처럼 무르게 만들어(스프링의 단단함 K(강성) 를 작게) 밀리면 살짝 물러나게 하는 **임피던스 제어**도 있다. 이름만 알아 둔다.

대학 · 수식

막힌 뒤 계속 누르는 토크: $\tau = u_제한 - m\,g\,l\,\sin\theta_c$ ($u_제한$: 토크 제한, θ_c: 닿은 각도)

닿은 점에서의 힘: $F = \tau / r$ (r: 관절에서 닿은 점까지 거리)

토크 제한 = 작업에 필요한 최대 토크 + 작업 여유 (예: 4-1 의 한계 = 필요 × 1.2)

안전 여유 = 몸 부위의 힘 한계 − 누르는 힘 (힘 한계는 ISO/TS 15066)

임피던스 제어(이름만): $u = K\,(\theta_목표 - \theta) + D\,(\theta_목표 - \theta)$ (K 가 작을수록 무르다)

탐구 활동 시뮬레이터 [교육] 탭 · 5-4 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 제한 3 N·m — 처음 조건 · 토크 제한: 3 N·m

목표	**토크 제한**이 클 때 팔이 손에 막히면 손을 얼마나 세게 누르는지 잰다.
예상	팔이 40° 에서 손에 막혀 목표 60° 에 못 가면, 제어기는 힘을 줄일까, 계속 키울까? ☐ 1. 바로 힘을 끈다 ☐ 2. 힘을 줄인다 ☐ 3. 제한값까지 계속 키운다
할 일	토크 제한 을 3 N·m 로 두고 올리기 를 누른다.
볼 것	팔이 40° 에서 손에 막힌 뒤에도 계속 밀어, 손을 누르는 토크가 약 2.6 N·m(누르는 힘 약 8.3 N)로 크다
왜	막힌 뒤에도 목표까지 오차가 남아 있어 PID 는 힘을 계속 키우고(I 항이 쌓임), 결국 모터는 제한값 3 N·m 까지 낸다. 그중 약 0.40 N·m 는 그 자리에서 팔 무게를 버티는 데 쓰이고, 나머지 약 2.6 N·m 가 손을 누른다. 관절에서 닿은 점까지 $r_{\text{팔 길이}}$ 이 약 0.31 m 이므로 $F_{\text{누르는 힘}}$ = 2.6 ÷ 0.31 ≈ 8.3 N 이다.
핵심	**제한 3 N·m → 약 2.6 N·m 로 세게 누른다**

2 활동 2 · 제한 1.5 N·m — 바꾸는 것 · 토크 제한: 3 N·m → 1.5 N·m

목표	**토크 제한**만 낮추면 누르는 토크가 얼마나 줄고, 도착 시간은 어떻게 되는지 확인한다.
예상	제한을 3 → 1.5 N·m 로 절반으로 낮추면, 손을 누르는 토크도 딱 절반이 될까, 그보다 더 줄까? ☐ 1. 절반보다 더 준다 ☐ 2. 딱 절반이 된다 ☐ 3. 그대로다
할 일	토크 제한 만 1.5 N·m 로 낮춘다. 같은 올리기 를 누른다.
볼 것	같은 자리에서 막히고, 누르는 토크가 약 1.1 N·m(약 3.5 N)로 절반 아래가 된다. 손이 없을 때 60° 에 자리 잡는 시간은 약 1.5 초로 조금 늦어진다
왜	누르는 토크는 제한에서 중력 토크 0.40 N·m 를 뺀 값이라 1.5 − 0.4 = 1.1 N·m 로, 절반(1.3 N·m)보다 더 준다. 팔 무게를 버티는 몫은 그대로이고 손을 누르는 몫만 줄기 때문이다. 손이 없을 때는 가속에 쓸 토크가 조금 줄어, 60° ±2% 안에 자리 잡는 시간이 약 1.1 초에서 약 1.5 초로 조금 늦어진다.
핵심	**제한 1.5 N·m → 약 1.1 N·m 로 절반 아래**

3 활동 3 · 제한 0.6 N·m — 바꾸는 것 · 토크 제한: 1.5 N·m → 0.6 N·m

목표	버티는 데 필요한 토크 가까이까지 제한을 낮출 때 얻는 것과 잃는 것을 함께 본다.
예상	제한을 0.6 N·m 로 낮추면 손을 누르는 토크는 얼마쯤일까? 손이 없을 때 60° 까지 가는 시간은 어떻게 될까? ☐ 1. 약 0.6 N·m, 도착 그대로 ☐ 2. 약 0.2 N·m, 도착 늦어짐 ☐ 3. 약 0.2 N·m, 도착 빨라짐
할 일	토크 제한 만 0.6 N·m 로 낮춘다. 같은 올리기 를 누른다.
볼 것	누르는 토크가 약 0.2 N·m(약 0.6 N)로 거의 안 누른다. 대신 손이 없을 때 60° 에 자리 잡는 시간이 약 2.6 초로 늘어난다
왜	누르는 토크는 0.6 − 0.4 = 0.2 N·m(약 0.6 N)로 아주 작다. 하지만 60° 에서 버티는 데만 0.54 N·m 가 들어, 60° 가까이에서는 가속에 쓸 토크가 0.06 N·m 쯤밖에 남지 않는다. 그래서 손이 없어도 천천히 올라가 60° 에 자리 잡는 데 약 2.6 초가 걸리고, 짐을 조금만 달아도 60° 에서 버티지 못한다.
핵심	**제한 0.6 N·m → 거의 안 누르지만 도착 2.6 초**

✎ 기록표: 화면의 결과를 적어요

활동	토크 제한(N·m)	손을 계속 누르는 토크(N·m)와 힘(N)	손이 없을 때 60° ±2% 안에 자리 잡는 시간(s)
활동 1 제한 3 N·m			
활동 2 제한 1.5 N·m			
활동 3 제한 0.6 N·m			

정리: 세 활동 비교하기

활동 1·2·3의 누르는 토크를 나란히 놓는다. 손이 없을 때의 도착 시간도 함께 놓는다. 안전(덜 누름)과 성능(빨리 감)의 균형을 본다.

알게 된 것:

정리 질문

토크 제한은 얼마로 정해야 할까?

내 답:

기억할 것

토크 제한을 낮추면 닿았을 때 덜 누른다. 하지만 일을 못 할 수도 있다. 그래서 제한은 필요한 만큼에 작은 작업 여유를 더해 정한다.

현장에서는

협동 로봇은 울타리 없이 사람 곁에서 일한다. ISO/TS 15066 에는 힘 · 압력 한계가 있다. 몸 부위마다 정리해 둔 값이다. 협동 로봇은 그 한계 안에 들도록 힘과 속도를 제한한다. 관절 토크나 모터 전류가 예상보다 커질 때가 있다. 그러면 충돌로 보고 멈추거나 물러난다. 그래서 사람과 함께 일하는 구간에서는 속도를 낮춘다. 울타리 안에서 일할 때보다 느리게 두는 것이 보통이다.

교사용 답

예상 정답: 활동 1: 3. 제한값까지 계속 키운다 · 활동 2: 1. 절반보다 더 준다 · 활동 3: 2. 약 0.2 N·m, 도착 늦어짐

정리 질문: 작업에 필요한 토크에 작은 작업 여유를 더한 값으로 정한다. 그래야 누르는 힘이 사람 몸의 한계보다 넉넉히 작다.

개념 확인 질문: 토크 제한을 낮추면 닿았을 때 누르는 힘은 어떻게 될까?

다음 차시 · 19차시 5-5 오차와 캘리브레이션: 같은 명령을 보내도 실제 로봇은 조금 다른 곳에 간다. 왜 시뮬레이터와 다를까?

협동 로봇 Collaborative robot (cobot) — 울타리 없이 사람 곁에서 일하도록 힘·속도를 제한한 로봇

토크 제한 Torque limiting — 제어기가 모터에 내는 토크에 상한을 두는 것. 닿았을 때 누르는 힘이 그만큼으로 묶인다

안전 여유 safety margin — 닿기 직전이 아니라 앞에서 멈추거나 약하게 누르도록 남겨 두는 여유. 거리나 힘(몸이 견디는 한계 − 누르는 힘)으로 잰다.

임피던스 제어 Impedance control — 손끝을 스프링·댐퍼처럼 무르게 만들어 밀리면 물러나게 하는 제어. 이름만

힘·파워 제한 Power and force limiting — 사람과 닿아도 다치지 않도록 로봇의 힘과 속도(파워)를 제한하는 협동 운전 방식

작업 여유 Working margin — 토크 제한을 작업에 필요한 토크보다 조금 크게 둔 차이. 크면 일은 잘하지만 닿았을 때 더 세게 누른다

충돌 감지 Collision detection — 토크나 오차가 갑자기 커진 것을 보고 부딪힌 것으로 판단해 멈추거나 물러나는 기능

19차시 5-5 오차와 캘리브레이션

Error and Calibration · 단원 안 5 / 5차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

정확도와 **반복 정밀도**의 차이를 말할 수 있다. 예를 들어 설명할 수 있다.

$\delta\theta_{\text{영점 오차}}$ 는 손끝에서 $r_{\text{팔 길이}} \times \delta\theta$ 로 커진다. 그래서 축에서 먼 점일수록 많이 빗나간다. 그 까닭을 계산으로 보일 수 있다.

알려진 점에서 잰 값으로 **영점 오차**를 찾는다. 찾은 오차를 명령에서 뺀다. 이 **캘리브레이션** 순서를 말할 수 있다.

Q 생각 열기

같은 명령을 보내도 실제 로봇은 조금 다른 곳에 간다. 왜 시뮬레이터와 다를까?

함께 생각해 보기

다트가 한곳에 모였는데 과녁 가운데가 아니라면 무엇이 문제일까?

저울의 0 이 어긋나 있으면 어떻게 고칠까?

해 볼 것

- ☐ 과녁 그림에 "모였지만 빗나감" 과 "흩어졌지만 가운데" 를 점으로 그려 본다
- ☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

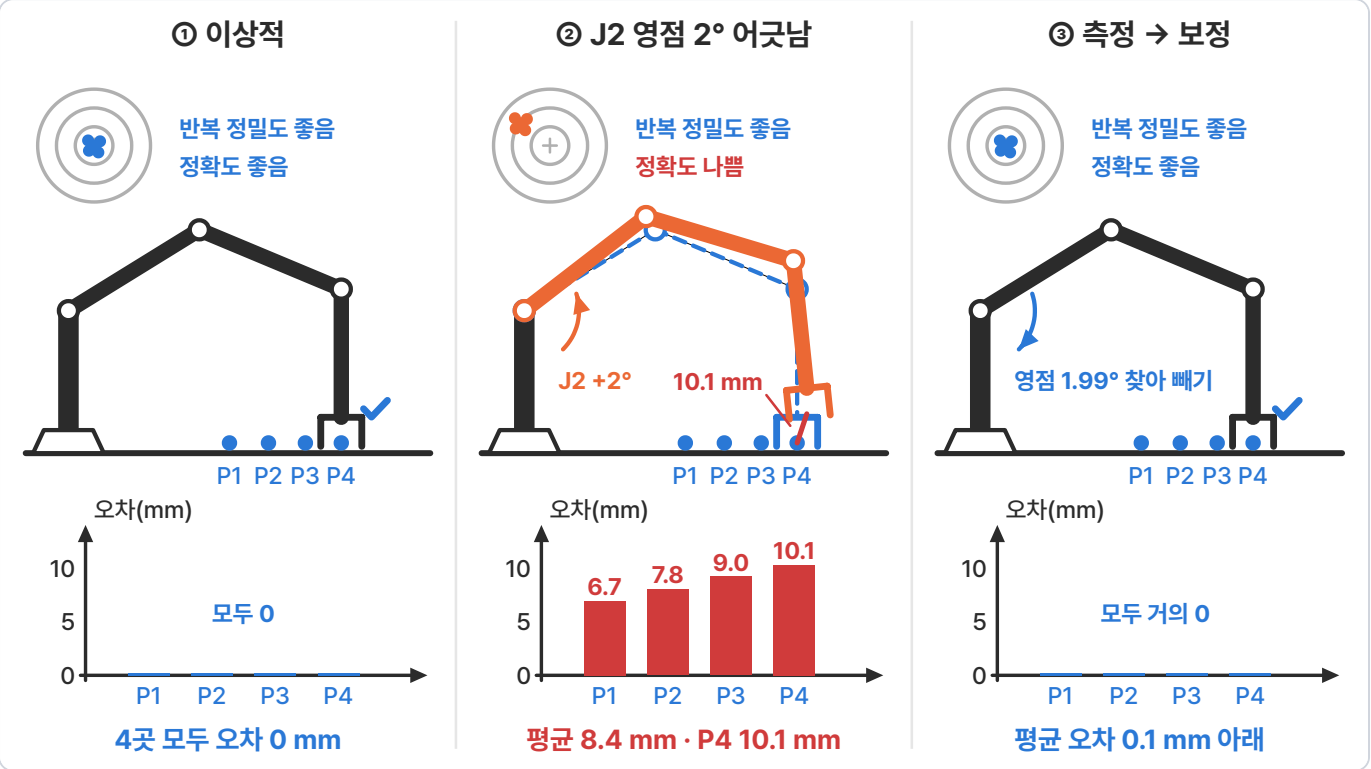


그림 5-5 J2 영점 오차가 만드는 손끝 오차와 보정: 이상적 · 영점 2° 어긋남 · 측정 후 보정 (P1~P4)

정의

관절의 영점(0° 기준)이 모델과 조금 다를 수 있다. 링크 길이도 조금 다를 수 있다. 그러면 손끝이 빗나간다. 알려진 점에서 실제 위치를 재서 모델 값을 고친다. 이것을 캘리브레이션(보정)이라고 한다.

원리

모델과 실제가 같으면 명령한 자리에 정확히 닿는다. **오차**는 0 mm 다.

영점이 조금만 어긋나도 모든 점에서 빗나간다. 팔을 멀리 뻗은 점일수록 많이 빗나간다.

알려진 점 몇 곳에서 빗나간 거리를 잰다. 그러면 **영점 오차**를 계산해 명령에서 빼 줄 수 있다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
$\delta\theta$	델타 세타	**영점 오차**: 관절의 0° 기준이 어긋난 작은 각도	° (식에서는 rad)
r	알	**팔 길이**: 회전축에서 손끝까지 거리	mm
Δx	델타 엑스	**손끝 오차**: 손끝이 목표에서 빗나간 거리	mm
J	제이	**자코비안**: 관절을 조금 돌릴 때 손끝이 어느 쪽으로 얼마나 움직이는지 적은 표	mm/rad

개념 자세히

시뮬레이터에서는 명령한 각도와 실제 각도가 같다. 실제 로봇은 다르다. 관절의 0° 기준(**영점**)이 조금 어긋나 있거나 링크 길이가 설계와 조금 다르면, 같은 명령을 보내도 손끝이 빗나간다. 명령한 위치에 얼마나 가깝게 가는지를 **정확도**, 같은 명령을 반복할 때 도착점이 얼마나 모이는지를 **반복 정밀도**라고 한다. myCobot 280 의 반복 정밀도는 ±0.5 mm 로 공개되어 있지만, 정확도는 모델이 실제와 다르면 그보다 훨씬 나쁠 수 있다.

각도 **오차**는 팔 끝으로 갈수록 커진다. 손끝이 회전축에서 $\overset{\text{팔 길이}}{r}$ 만큼 떨어져 있으면 **영점** 오차 $\overset{\text{영점 오차}}{\delta\theta}$ 가 만드는 빗나감은 약 $r \times \delta\theta$ 다($\delta\theta$ 는 라디안, 1° ≈ 0.0175 rad). J2 영점이 2° 어긋나면, 축에서 약 190 mm 인 P1 은 6.7 mm, 약 290 mm 로 가장 멀리 뻗은 P4 는 10.1 mm 빗나가 평균 8.4 mm 다. 1° 만 어긋나도 250 mm 끝에서는 4 mm 넘게 빗나가므로, **반복 정밀도** ±0.5 mm 보다 훨씬 크다.

캘리브레이션은 알려진 점 몇 곳에 손끝을 보내 실제 위치를 재고, 측정과 모델의 차이가 가장 작아지는 **영점** 값을 계산해 고치는 것이다. 차이의 제곱을 모두 더한 값이 가장 작아지는 값을 찾으므로 **최소제곱**이라고 한다. 시뮬레이터에서 점 4곳을 ±0.2 mm 정도로 재면 영점 **오차**를 약 1.99° 로 찾아내고, 이만큼 J2 명령에서 빼고 보내면 평균 오차가 0.1 mm 아래로 준다. myCobot 은 관절을 0° 표시에 맞추고 영점을 다시 기록하는 기능(set_servo_calibration)을 제공한다.

캘리브레이션은 시뮬레이터와 실물을 맞추는 첫걸음이다. 실물 로봇이 연결돼 있으면 같은 점으로 보내 시뮬과 실물 손끝의 차이를 재 볼 수 있다. **영점**·길이뿐 아니라 마찰·질량 같은 동역학 값도 실제와 다르며, 이 sim-to-real 차이는 7 로봇 비전 · 인식의 손-눈 보정과 8 로봇 학습에서 다시 만난다.

대학 · 수식

작은 각도 오차가 만드는 손끝 오차: $\Delta x \approx J(\theta)\,\delta\theta$ (J: 자코비안)

호 길이: $\Delta = r \cdot \delta\theta[\text{rad}]$ ($250\text{ mm} \times 1° \approx 4.4\text{ mm}$, $300\text{ mm} \times 2° \approx 10\text{ mm}$)

보정(최소제곱): $\delta^* = \operatorname{argmin} \Sigma \, \| x_{\text{측정},i} - FK(\theta_i + \delta) \|^2$

보정한 명령: $\theta_{\text{명령}} = \theta_{\text{원하는 각도}} - \delta^*$

탐구 활동 시뮬레이터 [교육] 탭 · 5-5 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 이상적 — 처음 조건 · J2 영점 오차: 0°

목표	모델과 실제가 같을 때 손끝이 목표 점에 얼마나 정확히 닿는지 확인해 비교의 기준을 만든다.
예상	시뮬레이터의 모델과 실제 팔이 똑같다면, P1~P4 네 점의 **오차**는 몇 mm 일까? ☐ 1. 0 mm ☐ 2. 약 1 mm ☐ 3. 약 10 mm
할 일	4곳으로 보내기 를 눌러 손끝을 P1~P4 로 보낸다.
볼 것	모든 점에 **오차** 0 mm 로 정확히 닿는다
왜	역기구학으로 구한 명령 각도를 팔이 그대로 따르고 **영점** · 링크 길이도 모델과 같으므로, 손끝이 계산한 자리에 정확히 간다 (**오차** 0 mm). 실제 로봇은 이렇게 되지 않으므로, 이 결과가 활동 2·3을 견주는 기준이 된다.
핵심	**이상적 모델 → 4곳 모두 오차 0 mm**

2 활동 2 · 영점이 어긋나면 — 바꾸는 것 · J2 영점 오차: 0° → 2°

목표	J2 **영점**이 2° 어긋나면 손끝이 점마다 얼마나 빗나가는지 잰다.
예상	J2 **영점**이 2° 어긋나면 네 점 가운데 어느 점이 가장 많이 빗나갈까? 몇 mm 쯤일까? ☐ 1. P1, 약 1 mm ☐ 2. 네 점 모두 똑같이 2 mm ☐ 3. P4, 약 10 mm
할 일	J2 영점 오차 슬라이더를 2° 로 둔다. 같은 4곳으로 보내기 를 누른다.
볼 것	실제 팔(주황)이 모든 점에서 빗나가고(평균 8.4 mm), 가장 멀리 뻗은 P4 가 가장 많이(10.1 mm) 빗나간다
왜	**영점 오차**는 모든 명령에 똑같이 2°(약 0.035 rad)를 더하지만, 손끝이 움직이는 거리는 회전축에서의 거리 $r^{\text{팔 길이}}$ 에 비례한다($r \times \delta\theta^{\text{영점 오차}}$). 축에서 약 190 mm 인 P1 은 6.7 mm, 약 290 mm 인 P4 는 10.1 mm 빗나가 먼 점일수록 크다. 이 오차는 **반복 정밀도**(±0.5 mm)보다 열 배 넘게 크지만, 매번 같은 쪽으로 같은 만큼 생기므로 재서 고칠 수 있다.
핵심	**영점 2° → 평균 8.4 mm, P4 가 가장 크다**

3 활동 3 · 측정해서 보정 — 바꾸는 것 · 보정: 안 함 → 측정해서 보정

목표	빗나간 위치를 재서 **영점 오차**를 계산하고, 보정하면 오차가 얼마나 줄어드는지 확인한다.
예상	네 점에서 잰 손끝 위치만으로 숨어 있는 **영점 오차** 2° 를 알아낼 수 있을까? 보정하면 평균 오차는 얼마나 줄까? ☐ 1. 알아낼 수 없다 ☐ 2. 약 1.99° 로 찾아 오차가 거의 없어진다 ☐ 3. 찾아도 오차는 그대로다
할 일	2° 그대로 측정 을 누른다. 그다음 보정 계산 을 누른다.
볼 것	계산한 **영점 오차**가 약 1.99° 로 실제 2° 와 거의 같고, 이만큼 빼고 보내니 평균 오차가 0.1 mm 아래로 준다
왜	**영점 오차**가 정해지면 네 점이 빗나가는 방향과 거리가 정해지므로, 거꾸로 잰 빗나감에 가장 잘 맞는 $\delta\theta$ 를 찾으면 영점 오차를 알 수 있다(**최소제곱**). 측정에 ±0.2 mm 잡음이 있어 약 1.99° 로 찾았고, 이것을 J2 명령에서 빼면 남은 영점 오차가 0.01° 쯤이라 평균 오차가 0.1 mm 아래로 준다.
핵심	**보정 → 영점 1.99° 를 찾고 0.1 mm 아래로**

✎ 기록표: 화면의 결과를 적어요

활동	점별 오차(mm)	평균·최대 오차(mm)	추정한 영점 오차(°)
활동 1 이상적			
활동 2 영점이 어긋나면			
활동 3 측정해서 보정			

정리: 세 활동 비교하기

점별 오차 막대를 P1~P4 로 나란히 놓는다. 이상적 · 영점 오차 · 보정 후를 비교한다. 먼 점의 큰 오차가 보정으로 사라지는지 본다.

알게 된 것:

정리 질문

실제 로봇을 시뮬레이터와 맞추려면 무엇을 해야 할까?

내 답:

기억할 것

작은 각도 오차가 팔 끝에서 커진다. 정확도는 재서 보정으로 고친다. 반복 정밀도는 기계가 정한다.

현장에서는

공장에서는 로봇을 설치한 뒤 영점을 다시 잡는다. 모터 · 감속기를 바꾼 뒤에도 그렇다. 관절을 기준 표시에 맞춰 잡는다. 컴퓨터 시뮬레이션에서 동작을 만들기도 한다. 이것을 오프라인 프로그래밍이라고 한다. 이 동작을 실물에 그대로 쓰려면 보정해야 한다. 레이저 트래커 같은 장비로 손끝을 여러 곳에서 잰다. 그리고 영점 · 링크 길이 같은 모델 값을 고친다. 이것이 로봇 캘리브레이션이다.

교사용 답

예상 정답: 활동 1: 1. 0 mm · 활동 2: 3. P4, 약 10 mm · 활동 3: 2. 약 1.99° 로 찾아 오차가 거의 없어진다

정리 질문: 알려진 점에서 손끝 위치를 재서 영점 오차를 계산하고, 그만큼 명령에서 빼는 캘리브레이션을 한다.

개념 확인 질문: 정확도와 반복 정밀도는 무엇이 다를까?

다음 차시 · 20차시 6-1 충돌 검사: 로봇은 가는 도중 어디에 부딪힐까? 움직이기 전에 어떻게 미리 알까?

영점
zero position (calibration)
각 관절의 0° 기준. 어긋나면 모든 위치가 조금씩 틀어진다.

오차
error
목표와 지금 값의 차이.

정확도
Accuracy
명령한 위치에 얼마나 가깝게 가는지

반복 정밀도
repeatability
같은 곳으로 여러 번 보냈을 때 도착점이 얼마나 모이는지.

캘리브레이션
Calibration
알려진 점에서 실제 위치를 재서 영점·길이 같은 모델 값을 고치는 일 (보정)

최소제곱
Least squares
측정과 모델의 차이를 제곱해 더한 값이 가장 작아지는 값을 찾는 방법